SESSION-REPORT

Session Report — Lumen / setup-agents-wizard hardening

Written for a reader who was not present. Every number below was either measured by the author of this report during the window stated, or is cited to the artifact that carries it. Where the session ledger (`.superpowers/sdd/progress.md`) makes a claim that could not be corroborated, this document says so rather than repeating it.

Repository	`<ext-volume-host>/Projects/vasic`
HEAD at time of writing	`62706a9` — `git ls-remote github main` returns the same SHA, so local == remote
Independent measurement window	**2026-08-27 18:21Z – 18:29Z** (20:21–20:29 CEST)
Primary source	`.superpowers/sdd/progress.md` (~830 lines, written as the work happened)
Method	Ledger read first, then each load-bearing claim replayed against commits, scripts, docs and live state

> **The repository was not quiescent while this was written.** `.github/workflows/ci.yml`, `scripts/setup-agents-wizard.sh` and `scripts/test-setup-agents-wizard.sh` were all modified by another actor at 20:05–20:06 CEST, mid-session. See §5.6.

1. Executive summary

What was asked. Put Lumen on `PATH` properly; extend and harden `scripts/setup-agents-wizard.sh`; cover it with tests and machine-readable evidence; add backup/rollback; document it; commit and push; sync the code indexes. The scope grew during the session to include constitution adoption, a telemetry opt-out, an embedding-backend fault investigation, and the removal of hardcoded machine paths.

What was delivered.

- A hardened wizard, a manifest-based rollback tool, and a **147-assertion** test suite across ten groups — 147 PASS, 0 fail, 0 skipped, in the newest evidence run.
- Three operational scripts promoted out of scratch space (`ollama-vulkan-remediation.sh`, `lumen-reindex.sh`, `lumen-index-doctor.sh`), plus a repository-hygiene gate (`audit-hardcoded-paths.sh`).
- Root cause found, and fixed on the host, for a GPU fault that had been **silently corrupting stored embedding vectors for roughly five weeks**.
- Four constitution pointer carriers at the root, propagation applied to five owned submodules across ten push URLs on four providers, and a 58-gate validation sweep.
- Every hardcoded `<macos-host>/Projects/vasic` path removed from executable code (18 files, 33 occurrences) behind a mutation-proven audit guard.
- 18 documents under `docs/setup-agents-wizard/` and `docs/constitution-adoption/`, including two adversarial verification reports left deliberately unedited.

What remains open. The full Lumen rebuild is still running after **10 h 40 m** and has made no measurable forward progress in the last 7½ minutes of observation; **169 corrupt vectors survive in the live index**; the constitution sweep sits at **37 PASS / 21 FAIL** with no umbrella-side change able to move it; three open conflicts (OC-1/2/3) await an operator ruling; and a substantial uncommitted workstream (a CI-workflow audit and a residual-paths audit) is in flight and is **not covered by the ledger at all**.

---

2. The defects found, ranked by consequence

These are not equals. The first destroyed data silently for weeks. The last is a cosmetic `--help` bug. They are ordered by what would have happened had nobody looked.

2.1 — The ollama/Vulkan silent-corruption chain *(catastrophic; data loss, undetected for ~5 weeks)*

Symptom. Lumen indexing runs kept dying with `HTTP 500 {"error":"failed to encode response: json: unsupported value: NaN"}` — and Lumen prints its *usage text* after the error, so it read as CLI misuse. A test "flake" (F6) and a chained reindex failure were both this, misattributed.

Root cause. `ollama 0.23.4` ran `library=Vulkan` on an Intel Iris Xe iGPU. A large embedding dispatch exceeds the i915 fence-wait timeout (~20 s); the kernel logs `Fence expiration time out` and abandons the dispatch — **and ollama reads the result buffer anyway**. Documented in `docs/setup-agents-wizard/OLLAMA-NAN-WEDGE.md` §4.1. The first hard NaN-500 landed at **request 8 in both** full-length reproductions; CPU passed the byte-for-byte identical payload 3/3. `OLLAMA_VULKAN=0` and `OLLAMA_LLM_LIBRARY=cpu` are both no-ops in this build; only `GGML_VK_VISIBLE_DEVICES=-1` yields `library=cpu`.

Four failure modes, worst last (`OLLAMA-REMEDIATION.md` §3):

#	What comes back	HTTP	Caught by a per-vector check?
1	`{"error":"… unsupported value: NaN"}`	500	Yes — loud
2	An all-zero vector	**200**	Yes — a zero-norm test sees it
3	Runner wedges; every later request returns NaN	500	Yes, but usually blamed on the caller
4	**A repeated STALE vector** — well-formed,	**200**	**No. Invisible to every per-vector test.**

#	What comes back	HTTP	Caught by a per-vector check?
	768 dims, unit norm		

Mode 4 is the one that mattered, and it is why the loud NaN was a red herring: the loud failure aborts a run, it does not store garbage. The silent one stores garbage that looks perfect.

Evidence. `INDEX-CORRUPTION-RECONCILIATION.md`: **758 of 35,717 stored vectors (2.12%) were byte-identical to one another while representing 695 distinct texts across 55 files**, in a perfectly contiguous write range (`vec_chunks_rowids.rowid` 28205–28962, zero gaps). `_content_fa/` was 508/508 corrupt; `_content_es/` 250/458. They pass everything: not NaN, not Inf, not all-zero, L2 norm 1.000000083. Proven semantically — the corrupt vectors score **0.25–0.47** cosine against their healthy `en/de/fr/ar` counterparts where the healthy cross-language baseline is **median 0.967**.

The near-miss inside the near-miss. An earlier full forensic audit, `LUMEN-INDEX-INTEGRITY.md`, concluded *"The index is **NOT CORRUPT**… **Do NOT rebuild it.**"* Every number in that report is correct and reproduces exactly. Its **inference** did not follow. Its one test that would have caught this — a distinctness check — was run on **blocks 1–4 only (4,096 of 35,717 vectors)**. The corruption lived in blocks 28–29. Acting on that report's recommendation would have preserved the corruption permanently, because a corrupted file still carries a non-empty content hash and an incremental run skips it forever.

Fix. `GGML_VK_VISIBLE_DEVICES=-1` written to `/etc/sysconfig/ollama` (which the unit already sources via `EnvironmentFile=-`), applied by the operator with `sudo`, service restarted **2026-08-27 09:44:18 CEST**. Verified by me at 18:2xZ: `library=cpu`, **0 i915 faults since that restart** (520 in the preceding 24 h are pre-remediation history), and a 32-chunk batch probe returning **32/32 distinct vectors**. `scripts/ollama-vulkan-remediation.sh --check` exits 0.

Honest caveat carried from the source. `INDEX-CORRUPTION-RECONCILIATION.md` §8 item 4 states that the causal chain *"GPU fence timeout → stale buffer → repeated vector"* **is inference from correlation, not reproduction**, and explicitly cannot exclude *"a*

buffer-reuse bug in Lumen's own `embedBatch` response handling." The observable effect and the remedy are identical either way. This report does not upgrade that inference to a proof.

A related falsified hypothesis, worth recording because it was the obvious one. Capping chunk size would **not** have prevented this. Chunk sizes are tiny — median 99 chars, p99 1,941, max 6,065, **zero over 8,000**. Lumen batches **32 chunks per request**, and the operative quantity is the batch total: the corrupting requests carried **16,698–31,364 characters each**, and **244 of 1,107 requests (22%)** exceeded the 12,800 chars ever validated as safe. `OLLAMA-NAN-WEDGE.md` still carries its now-falsified "cap chunk size under ~4000 characters" recommendation; the correction lives only in the two later documents.

2.2 — `_tools/deploy-langs.sh` failed silently on a macOS path, then committed and pushed *(severe; a live site deploy running from an arbitrary directory)*

Symptom. None. That is the defect.

Root cause. Verified verbatim from `git show 72dc135^:_tools/deploy-langs.sh`:

```
set -uo pipefail                        # note: no -e
ROOT="<macos-host>/Projects/vasic"     # the original author's
        macOS box
cd "$ROOT"                              # no ||, no guard
```

Without `-e` the failed `cd` did not abort. The script therefore ran **from the caller's working directory** with `GEN` and `PDF` pointing at nonexistent paths — and then went on, at lines 89–103, to `git -C "$s" commit` and `git -C "$s" push "$r" main` for **both** site submodules (`vasic.digital` and `milosvasic.ru`, both registered in `.gitmodules`).

Fix. `ROOT` derived from the script's own location and the `cd` made fatal:

```
ROOT="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT" || { echo "FATAL..." >&2; exit 1; }
```

Evidence quality is worth noting. The ledger records that two verification attempts were themselves invalid — `BASH_SOURCE` cannot be faked via `bash -c` or process substitution — and that only tracing the **real** script with `bash -x`, invoked from `/tmp`, proved the fix. That is the correct standard and it was applied.

`_tools/watch-deploy.sh` carried the same defect and was the more dangerous of the two. Its before-state is a bare `cd <macos-host>/Projects/vasic` under `set -uo pipefail`, followed by `for i in $(seq 1 240); do ... bash _tools/deploy-langs.sh ... sleep 1200`. That is **240 cycles × 20 minutes ≈ 80 hours** of a watcher running in the caller's directory, invoking a relative `_tools/deploy-langs.sh` that does not exist there. It now derives `ROOT="${VASIC_ROOT:-$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.." && pwd)}"` with a fatal `cd`.

2.3 — The near-miss: a correct path fix alone would have triggered a mass PAID re-translation *(severe; direct financial cost, and the trap was invisible)*

Symptom. Would have been a large, unexplained bill and 532 overwritten translations.

Root cause. 537 review JSONs under `_tests/evidence/translate-new/` bake `"_translated": "<macos-host>/Projects/vasic/_content_<lang>/..."` **into the data** — I confirmed **523** of them carry such a path, exclusively under the `_translated` key. Three scripts gate on `os.path.isfile()` of that recorded absolute path: `_tools/translate/matrix.py` (via its `anchored()` helper) and inline Python in `_tools/translate/run-batch.sh` and `_tools/translate/finalize-review.sh`. The state machine is:

State	Behaviour
Before any fix	broken root → `isfile()` false → silent no-op
Root fix alone	scripts now run for real → docs judged *"not PASS"* → **mass re-translation against PAID APIs**
Correct fix	re-anchor the repo-relative tail onto the derived repo root, **only** when the recorded absolute path is missing, and **only** if the re-anchored file exists

> **Correction to the ledger's figure.** The ledger states the root-fix-alone path *"would have RE-TRANSLATED all 532 (lang, doc) pairs."* Measured directly by read-only simulation over the 38 × 14 matrix: with re-anchoring, **532 PASS**; without it, **10 PASS → 522 would have been judged not-PASS**. The 10 survivors

> recorded a *relative* `_translated` value, which resolves only because cwd is the repo root. So the near-miss is **522** paid re-translations, not 532. The mechanism and the order of magnitude are right; the number is off by ten.

This is the most instructive defect of the session: **fixing the bug correctly, and stopping there, would have been worse than leaving it broken.** The no-op was load-bearing. The guard against faking a PASS — requiring the re-anchored file to actually exist — is what makes the fix safe. The guard reads, in `matrix.py:27-38`: *"This never fakes a PASS: the re-anchored file still has to actually exist."*

Reproduced live during this report (both are read-only branches — no writes, no API calls):

```
python3 _tools/translate/matrix.py                    -> MATRIX
(docs=38, langs=14, pairs=532)
                                                         ALL 532 0
0 0      (532 PASS / 0 FAIL / 0 ERROR)
bash _tools/translate/finalize-review.sh --status ru  -> ru
current_PASS=38/38 stale_PASS=0

need_action=0 missing_translation=0
```

2.4 — The SuperSpec submodule was deleted on every wizard run *(severe; silent, repeated data loss)*

Symptom. A registered submodule vanished on each run, with the wizard reporting success.

Root cause. For a registered submodule, `.git` is a **file** (a gitlink), not a directory. Verified verbatim from `git show 6148de3^:scripts/setup-agents-wizard.sh`:

```
if [[ -d "$SUPERSPEC_PATH/.git" ]]; then          # ALWAYS false
        for a submodule
    print_success "SuperSpec already cloned."
else
    if [[ -d "$SUPERSPEC_PATH" ]]; then rm -rf "$SUPERSPEC_PATH";
        fi
```

The `-d` test can never be true for a gitlink, so the `rm -rf` fired on every single run, then replaced the submodule with a plain clone — corrupting the parent repository's submodule state.

Fix. `setup_superspec()` now tests with `-e`, and adds a `git -C "$root" submodule status` branch that runs `submodule update --init` instead of deleting. `rm -rf` survives only in the `else` branch, guarded by `[[ -d`

`"$sub_path" ]]`, for a genuinely stale non-git directory. Assertion `A6` pins the operator itself: `assert_contains "A6 submodule presence test uses -e (gitlink is a FILE)" '[[ -e "$sub_path/.git" ]]'`.

Regression cover. Group E is a genuine behavioural test, not a grep: it builds a throwaway parent repo, writes `submodules/superspec/.git` as a gitlink **file** plus an `IMPORTANT.txt` canary, sources the wizard in `SETUP_WIZARD_LIB_ONLY=1` mode, calls `setup_superspec`, and asserts the canary `SURVIVED`. Confirmed by me in the newest evidence file, all three green:

```
E1 REGRESSION: gitlink submodule is NOT deleted        [PASS]
E2 gitlink submodule reported as a submodule           [PASS]
E3 stale non-git directory is reported for removal     [PASS]
```

The first adversarial pass singled out `E1` as *"Load-bearing, and the strongest assertion in the suite."*

2.5 — Seven more silent wizard false-positives *(moderate; work reported as done that was never done)*

All eight original wizard bugs shared one shape: **a green summary line for work that did not happen.** Besides SuperSpec: the Lumen MCP used a non-existent `serve` subcommand (correct: `stdio`); four npm package names 404'd; two bare names (`kimi`, `mimo`) resolved to unrelated packages; agent MCP configs were written to paths no agent reads; `ToolCall` is not a Claude Code hook event; an unguarded `curl|bash` could abort the run under `set -e`; and the qwen binary is `qwen`, not `qwen-code`. Two further defects were found *by the new tests rather than by inspection* — `WIZARD_STATE_DIR` was frozen at source time, and `rollback --list` double-counted sessions through the `latest` symlink.

A ninth, of the same family, is worth naming separately because of its cause: **`jq`'s `//` operator returns the RHS when the LHS is `false`**, so the telemetry summary read a correctly-set `false` back as "unset" and reported "not disabled." Regression tests H7 and A32.

2.6 — The index doctor over-reports corruption by ~10× *(moderate; found by this report, not previously known)*

This one is new — found while writing this report. It is documented in full in **§5.5**, because it changes what the current headline numbers mean.

`scripts/lumen-index-doctor.sh` decodes **every** 768-float slot in every sqlite-vec block and never consults the **validity bitmap**. The index has 85 blocks × 1024 = **87,040 allocated slots** but only **52,516 live vectors**. The 34,524 dead slots hold freed and uninitialised data, and the doctor counts them as stored vectors. Its reported "largest identical group: 943" is **943 all-zero uninitialised slots** — not stored data at all.

The doctor's *verdict* is nonetheless correct: real corruption does survive. Its *magnitude* is not. This is the third instance in this session of the same underlying error — **scoping the vector population wrongly.** `LUMEN-INDEX-INTEGRITY.md` scoped too narrow (4,096 of 35,717) and declared the index clean; the doctor scopes too wide (87,040 of 52,516) and overstates the damage.

2.7 — Two `--help` defects *(trivial; cosmetic and one near-miss)*

`ollama-vulkan-remediation.sh --help` ran `sed -n '2,40p' "$0"` while its header ends at line 34, leaking **six extra lines** — `set -uo pipefail`, colour variables and three function bodies. Fixed to `sed -n '2,34p'`.

Filed alongside it because it was found in the same pass and is *not* trivial: `lumen-reindex.sh --help` **started a full index run**, and a `--forse` typo silently downgraded an intended rebuild to an incremental pass. Both fixed; `--forse` now exits 2, asserted behaviourally by `I15`.

---

3. What adversarial verification caught

Two independent passes were commissioned with the explicit goal of falsifying the engineer's claims. Between them they **refuted five claims** — three in pass 1 (`INDEPENDENT-VERIFICATION.md`), two in pass 2 (`INDEPENDENT-VERIFICATION-2.md`) — and found further defects. **Every refuted claim was a real bug in the engineer's own work**, including a source comment that was itself false.

3.1 The pattern

Listing the five refutations flattens them. They are almost all one failure mode:

> **An assertion that checks whether the code *mentions* something, instead of whether the code *does* something.**

A grep over a script's source text is satisfied by a function definition, a printed banner, a comment, or a TODO. It is satisfied by the *description* of the behaviour rather than the behaviour. Such an assertion is green before the feature is written, stays green when the feature is deleted, and is therefore worth less than no assertion at all — because it is *believed*.

Assertion	How the false-green worked	How it was repaired
A9	`assert_contains … "ensure_lumen"` over the whole file. The **function definition** satisfies the grep. Deleting the sole call site (`sed '713d'`) left it **PASS**.	Counts *call sites* with an anchored regex; `assert_eq … "1"`
A20	Hook rewritten to `ensure("BogusEvent")` and the test stayed green — the word `PreToolUse` survives inside a **`print_success` banner string**, which is executable and so not stripped by the comment filter. *A test kept green by the text of a success message.*	Counts `ensure\("(Pre|Post)ToolUse"\)` occurrences == 2, matching the jq writer, not a banner
A27	`assert_absent "codegraph index "` — a literal **with a trailing space**. `codegraph index` at end-of-line slips straight past it.	Regex `codegraph[[:space:]]+index([[:space:]]|$)`, count == 0
I21	`assert_contains "could not complete" "$dsrc"` greps the **doctor's source** and **runs the doctor zero**	Builds an index whose shadow tables are absent, runs the doctor, requires exit 2. Mutation-proven

Assertion	How the false-green worked	How it was repaired
	times. It was green while the behaviour it names was provably broken, and it stayed green when the entire guard was replaced by a `# TODO` comment. *It asserts that a comment exists.*	
C-2	Not a test, but the same shape: the wizard **printed the word "verified"** — `"(verified: no analytics strings in binary)"` — for a scan it never ran.	Now probes the resolved binary with `strings(1)` at runtime

Two further instances remain **open**, and this report does not paper over them:

- **`I11`** — `assert_contains "…REFUSING TO START"` greps the refusal branch without ever exercising the detector. Pass 2 notes it was *"green throughout the entire lifetime of R1, which is exactly how R1 shipped."* Still a source grep.
- **`I12`** — commented in the test file as *"Behavioural, not a grep"*, and described in `OPERATIONAL-SCRIPTS.md` and `MANUAL.md` as proven to *"actually reach `lumen index -f`"*. The implementation is **two source greps** and runs the reindexer zero times. This is the pattern with documentation asserting the opposite of the code.

Pass 2's own summary is the honest framing: **54 of 141 assertions (38%) were group-A static source greps**, and *"a suite can be 141-green and still miss a live defect in the thing it is testing."*

3.2 The two refutations that were self-inflicted fixes

Pass 2's two refutations are notable because both were defects *introduced while fixing something else*:

- **R3 — the exit-code fix was a no-op.** The doctor's contract is `0 healthy / 1 corruption / 2 could not inspect`. The guard written to enforce it was `case $rc in 0|1|2) exit $rc ;; *) … exit 2 ;; esac`, with a comment correctly stating *"python maps an unhandled exception to 1, which is our corruption code."* **And `1` was placed in the pass-through arm.** The comment described the bug; the code preserved it. A crash was still reported as "corruption found." Fixed properly in `fbccc7a` using private python exit codes (20/21/22) that bash translates, so no interpreter status can collide with a verdict.
- **R5 — `I21`**, above.

`R1`, `R2` and `R4` were real when commissioned and had been patched mid-pass; the verifier re-tested and confirmed the fixes. Pass 2 records the principle explicitly: *"a claim that was false when made is not retroactively true because it was fixed afterwards."*

3.3 The self-match family — and a caveat about its provenance

The ledger names **four occurrences** of one shape: a process filter whose own command line contains the pattern it searches for, so it matches itself.

1. A watcher using `pgrep -f "...index /.../vasic"` matched its own cmdline and **looped forever**.
2. `pkill -f "resilient-index.sh"` matched the author's own shell and **killed it** (exit 144), taking the monitor with it.
3. `A47`'s first draft matched command names inside **printed instruction strings** and `record_action` undo text — **7 false positives**.
4. The verification agent's own poller, `until ! pgrep -f probe2.py`, matched itself and reported *"still running"* about nothing but itself for ~10 minutes after the probe had finished.

The mitigation now in use is the bracket trick — `pgrep -f "resilient-index[.]sh"` — which prevents the pattern from matching its own literal text. (This report used the same trick for every process check it ran.)

Pass 2 itself names `R5` as the same family: *a check that looks like it observes behaviour but is really matching a string.* That is the through-line — **items 3 and 4 above are the same bug as `A9`, `A20` and `I21`, expressed against a process table instead of a source file.**

> ⚠ **Provenance caveat.** Occurrences 1, 2 and 4 are recorded **only in the ledger's own self-reported honesty section.** Neither verification report documents them — a search of both for `probe2`, `pgrep` and any self-match caveat returns nothing. They are therefore **uncorroborated by independent artifact**, and are reproduced here on the engineer's own testimony. Occurrence 3 *is* corroborated: the A47 false-positive filtering (`echo|print_*|record_action|check_command`) is present in the test file.
>
> Worth noting in mitigation: the project's own governance corpus carries the rule — `submodules/constitution/CLAUDE_ANCHORS_FULL.md` §11.4.174: *"Every pgrep/ps filter MUST exclude self-matches + other-project matches."*

---

4. The honesty ledger

Every claim that was stated and later corrected. **This section is the point of the document.**

#	What was claimed	What was actually true	How it was found
1	*"The index is **NOT CORRUPT**. It is TRUSTWORTHY… **Do NOT rebuild it.**"* (`LUMEN-INDEX-INTEGRITY.md`)	**758 vectors across 55 files were corrupt.** The report's numbers were all correct and reproduce exactly; the **inference** was wrong. Its distinctness check — the one test that would have caught it — ran on **blocks 1–4 only (4,096 of 35,717)**. The corruption was in blocks 28–29.	A follow-up reconciliation hashed **all** 35,717 vectors instead of a sample
2	*"NaN is absorbing under normalisation, so a tight unit-norm distribution*	True **and irrelevant**. A stale-but-well-formed vector has a perfect norm. Health is not a per-vector property.	Aggregate distinctness test: *N* distinct texts must give *N* distinct vectors

#	What was claimed	What was actually true	How it was found
	PROVES no NaN reached the writer” → index clean		
3	Chunk-size overflow is the trigger; *“cap chunks under ~4000 characters”*	**Falsified.** Max chunk 6,065 chars, zero over 8,000; largest chunk in the corrupt range 2,832. The trigger is the **32-chunk batch total** (16,698–31,364 chars). Capping chunk size would not have prevented it.	Measured the chunk-size distribution and reconstructed per-request batch totals
4	*“No summary line prints on mere file existence”* — asserted in a **source comment** at line 946	**False; four did** (launcher `-x`, completions `-r`, SpecKit `-d` on an empty dir, SuperSpec `-e` on an empty `.git`). The comment asserting otherwise was untrue.	Verifier replayed the four predicates against **zero-byte targets**: *“Four green checkmarks for four empty files”*
5	*“Only `codegraph sync` is invoked”*	**Half false.** `codegraph init` is invoked on the no-DB branch. (`codegraph index`, the destructive rebuild, genuinely is not.)	A single grep returning two lines
6	*“Each assertion tests what its name says”*	**False for 2 of 3 audited.** A9 matched a function definition; A20 was held green by a `print_success` banner.	**Mutation testing** — delete the call site, rewrite the hook, observe still-green
7	*“ CodeGraph index step finished.”*	Printed **unconditionally**, including after a failure — *“a warning and a green check for the same failed operation, one line apart.”*	Verifier read the branch structure
8	*“(verified: no analytics strings in binary)”*	A **hardcoded string**. No check ran.	Verifier looked for the scan and found none

#	What was claimed	What was actually true	How it was found
9	*“Nothing here mutates the real environment”* (test-suite header)	The suite writes `.test-evidence/<stamp>/` **into the repo**, and live mode runs real `lumen index`/`search`/`purge`.	Verifier inspected what the suite actually writes
10	*“Every probe is timeout-bounded”* (after the A47 fix)	**Half-fixed.** A47 covered `lumen search` **only**; unbounded `journalctl --since "-30 days"`, three `claude mcp` calls and `ollama list/ps` went unnoticed.	Pass 2 ran a whole-wizard grep instead of a section-scoped one
11	*“The doctor’s exit-code fix is in place”*	**A no-op.** `1` was placed in the pass-through arm of the very `case` written to intercept it. The comment was right; the code did the opposite.	Built an index with no shadow tables → traceback → `$? == 1`
12	*“`I21` proves the doctor maps crashes to 2”*	**Worthless.** It greps the doctor’s source and runs it **zero times**; stayed green with the entire guard replaced by a TODO comment.	Verifier deleted the guard and watched the test pass
13	*“No commit wrapper exists at this root”* (in all four root carriers, gap G5)	**False.** `commit "<msg>"` → `Upstreamable/commit.sh` → `Software-Toolkit/Utils/Git/commit.sh` → `push_all.sh` works here; a tracked `upstreams/GitHub.sh` exists. What is actually missing is git **hooks**.	The carriers’ own claim traced through the wrapper chain
14	*“No `scripts/verify-all-constitution-rules.sh`; the validation sweep does not exist”* (gap G3)	**False.** It exists and runs — 37 PASS / 21 FAIL / 0 ERROR of 58. Corrected to PARTIAL, naming the real gap (`verify-governance-cascade.sh` absent).	Running it

#	What was claimed	What was actually true	How it was found
15	*“No `helix-deps.yaml` at the repository root”* (gap G6)	**False.** It exists and parses. Corrected to CLOSED.	Looking
16	*“The carriers are 191 lines each”* (`Constitution.md`)	They were **197**. Replaced with a non-drifting instruction to verify by sha256 of line 24+.	Counting. *(Now 200 lines, sha24+ `961f47134720080e` on all four — verified by me)*
17	The Upstreamable **conditional pointer form** is the right model for the root carriers (`INVENTORY.md`)	It would have **FAILED all 17 propagation gates** at a repo root.	The carriers agent tested it before adopting it
18	The staged propagation is ready to apply	**Defective — it would have REGRESSED the gates.** Rev-1 drafts opened with a `>` blockquote instead of a non-fenced `## INHERITED FROM` heading, so each applied file registered as a **new anchor-less carrier**: 85 → **136** MISSING lines (§4.6 only), or 85 → **408** on a full apply — **4.8× worse**.	Measured in five temp roots with all 17 gates run, against a baseline control that reproduced 85/5 exactly
19	*“37 PASS”* means 37 enforced gates	**34 enforced + 2 honest SKIPs + 1 FALSE SKIP.** `cm_opendesign_ui_system.sh` exits 0 claiming *“no UI surface detected”*, which is false — eight real CSS files exist under `design-system/`. Its default `OD_*_GLOBS` never match this layout and the umbrella never binds them. Binding them would move	Read the gate’s globs against the actual tree

#	What was claimed	What was actually true	How it was found
		the sweep to an honest **36 PASS / 22 FAIL**.	
20	*“F5 completion count is 2”*	Asserted from a **loose grep**; the exact string appears once.	Self-caught
21	*“Index integrity verified clean, 0 corruption”* (stated in the ledger)	**Retracted by the ledger itself** — see row 1. Recorded here because the retraction is the point: the numbers survived, the conclusion did not.	Self-corrected
22	The `design-toolkit` push succeeded — the wrapper printed **`'github': OK`**	**It pushed nothing.** That repo has no `github` remote and no `upstreams/` recipe, so `push_all` walked **up** to the umbrella’s recipe and ran `git push github`, which does not exist there. Compounded by the repo being 1 commit behind a stale `origin/main`. Fixed by `git rebase origin/main` then `git push origin main` — **no force push**.	Noticed during the apply; **see the corroboration caveat below**
23	`INDEPENDENT-VERIFICATION-2.md` closes with R3/R5 *“still live”*	True when written, **fixed afterwards in `fbccc7a`**. Handled correctly: an ADDENDUM was added at the top rather than editing the findings, because *“rewriting an adversarial report after the fact would defeat the purpose of commissioning one.”*	Re-proved against the current scripts: 1 / 2 / 2 on corrupt / absent / crash
24	**`43 duplicate-vector groups; 1,783 vectors (2.05%); largest group 943; 1 all-zero`** — the	**Inflated ~10×.** Masked to the validity bitmap, the live index has **1 group of 169 vectors (0.32%)**, 0 NaN, 0 all-zero, 0 off-norm. The	**Found by this report** — see §5.5

#	What was claimed	What was actually true	How it was found
	doctor's current headline	"943" group is 943 **uninitialised dead slots**.	
25	*"The CI 'Bridge hardcoded config paths' step is DEAD CODE. **NOT removed** — `.github/` is inside the OC-1 governance conflict."*	**The step does not exist and has not for some time.** There is no `ln -s` anywhere in `.github/workflows/ci.yml`, at HEAD or in the working tree. It was removed by commit **`9aa95f2`**, which landed *before* `72dc135`; by `72dc135^` the workflow already carried *"the former / Volumes symlink bridge is no longer needed"*. What actually survived at HEAD is a **stale header comment** — the one containing *"Non-invasive and fully honest"* — describing a step that is gone.	**Found by this report**: searched `ci.yml` for the step and for `ln -s` at HEAD and in the tree, then bisected with `git log -S`
26	*"…would have RE-TRANSLATED all **532** (lang, doc) pairs"*	**522.** Ten of the 532 pairs recorded a *relative* `_translated` value and resolve regardless. Measured by read-only simulation. (The *"532 PASS"* verification figure is separately correct — that is the full matrix size.)	**Found by this report** — see §2.3

4.1 Self-inflicted process mistakes the ledger records against itself

These are not defects in the product. They are recorded because the ledger recorded them, and removing them would make this report less honest than its source.

- `| head -70` on the first wizard run sent SIGPIPE and killed it mid-Step-5.

- Probing the embedding backend with `ollama stop` **while the indexer was mid-round** corrupted the author's own diagnosis; at least one "still unhealthy after unload" was self-caused.
- **Committing while agents were still writing, three times over:** `f5626d6` swept in a propagation agent's in-progress drafts (clean by luck); `72dc135` swept in three throwaway `__trace_probe__` files (not clean — needed a follow-up delete); and tracing `watch-deploy.sh` regenerated `milosvasic.ru/sitemap.xml` and `vasic.digital/sitemap.xml` (525 lines each, pure `lastmod` bumps) which **nearly rode along into a live GitHub Pages deploy**. Reverted with `git checkout --`. The stated rule: *wait for every agent to report before running `commit`.*

4.2 Ledger claims this report could not corroborate

Stated explicitly rather than repeated as fact:

1. **The `design-toolkit` push incident (row 22) is documented nowhere in the repository.** A search of all of `docs/`, the root carriers, `Constitution.md` and `.ashlrcode/genome` for `push_all`, `'github': OK`, "incident", "rebase" and "force" returns nothing. `POST-APPLY-STATE.md` mentions `design-toolkit` 20 times, all about a different question, and records only the clean end state. **The premises are corroborated** — `design-toolkit` really has only an `origin` remote and really has no `upstreams/` directory — and **the fix is reconstructable from `git reflog`**, which shows `rebase (start): checkout origin/main` → `rebase (pick)` → `rebase (finish)` with no force push. But the false `'github': OK` itself is attested only by the ledger. **This is the single most operationally important lesson of the apply — "the commit wrapper is unsafe in a submodule that lacks its own `upstreams/` directory" — and it exists in no document that a future reader would find.**
2. **Three of the four self-match occurrences (§3.3) are uncorroborated** by either verification report.
3. **Throughput.** The ledger's close-out states *"Measured throughput 15 files/min (~900/hour) => roughly 2 more hours."* I could not reproduce this — see §6.1.
4. **The rebuild's denominator is unsettled.** The ledger says *"~3,800 files"*; `project_meta.total_files` reads **2,413**; `LUMEN-INDEX-INTEGRITY.md` says 3,833/3,835. The ~1,420-file discrepancy is noted in no document. "How much is left" is therefore between 16% and 47% depending on which figure is right.
5. **The CI symlink claim is refuted, not merely uncorroborated** — honesty-ledger row 25. The ledger says the *"Bridge hardcoded config*

paths" step is dead code that was deliberately left in place because `.github/` sits inside the OC-1 conflict. There is no such step. The real residue was a stale comment, and the concurrent actor has already replaced it and added a `Gate — hardcoded path audit` step that runs `./scripts/audit-hardcoded-paths.sh`.

6. Minor: `docs/setup-agents-wizard/README.md`'s index describes `INDEPENDENT-VERIFICATION-2.md` as *"1 refuted"*; the report itself records **2** (items 4 and 6, i.e. R3 and R5).

For balance, the ledger claims that *did* survive replay unchanged include: the eight original wizard bugs; the before/after source of `deploy-langs.sh`, `watch-deploy.sh` and the SuperSpec `rm -rf`; the *"18 files, 33 occurrences"* hardcoded-path count (reproduced exactly at `72dc135^` under the audit's own scope); the propagation temp-root measurements (85/5 → 136/8 → 408/24 → 68/4); the post-apply deltas (85→68 lines, 5→4 carriers, 11→31 pointer-skips) and all five submodule commit hashes; the 37/21/0-of-58 sweep result; the carrier lockstep sha `961f47134720080e`; and every number in the corruption forensics. The ledger is substantially reliable; the exceptions above are the exceptions.

---

5. Current verified state

All measured by me between **18:21Z and 18:29Z on 2026-08-27**, unless stated.

5.1 Test suite

Newest evidence: `.test-evidence/20260827T173006Z/summary.json` (written 19:36 CEST).


```
"totals": {"total": 147, "passed": 147, "failed": 0, "skipped":
        0}, "exit_code": 0
```


Confirmed independently from `results.tsv`: the status column is **147 × PASS**, nothing else. Group split:

Group	Assertions
A. Static analysis of the wizard	55
B. Lumen launcher (isolated `$HOME`)	8

Group	Assertions	
C. Shell configuration (isolated `$HOME`)	10	
D. MCP configuration (isolated `$HOME`)	5	
E. SuperSpec submodule safety (data-loss regression)	**3**	all PASS
F. Live integration (real environment)	11	including a real index + semantic search
G. Backup manifest and rollback	18	
H. Telemetry opt-out	7	
I. Operational scripts	22	
J. Hardcoded-path audit	8	
Total	**147**	

This is the first fully-green run **including** live integration: `F6/F7/F8` really did index a fixture and run a semantic search against the live backend, while the large rebuild competed for it. The two prior runs (`20260827T170932Z`, `20260827T164124Z`) were 143 total / 136 passed / **7 skipped**.

> The suite grew **96 → 103 → 117 → 122 → 126 → 134 → 143 → 147** during the session. Quote the figure with its evidence directory, not on its own.

5.2 Constitution sweep

`./scripts/verify-all-constitution-rules.sh` — **exit 1**.

```
SWEEP: FAIL — 21 FAIL + 0 ERROR out of 58 gate(s).
```

58 = 57 gates discovered under `submodules/constitution/scripts/gates/**` + this repo's `tests/test_constitution_inheritance.sh`. 58 − 21 − 0 = **37 PASS**, matching the documented record in `README.md` §5.3, `GATE-TRIAGE.md` §2 and `POST-APPLY-STATE.md` §1. §11.4.32 step 1 is an explicit **SKIP-with-reason** (`verify-governance-cascade.sh` absent) and is never counted as a pass.

> ⚠ **This measurement has since been overtaken.** `scripts/verify-governance-cascade.sh` was created (untracked) at 20:36, after this run. Step 1 will no longer be a SKIP. Re-run the sweep.

Read `37 PASS` as `34 enforced + 2 honest SKIPs + 1 false SKIP` — see honesty ledger row 19.

Separately verified green: `tests/test_constitution_inheritance.sh` → **8 PASS, 0 SKIP, 0 FAIL**, exit 0, including `I8 PROPAGATION-GATES — 17/17 ... PASS on the root carriers`. The four root carriers are **200 lines each** with identical `sha256(line 24+) = 961f47134720080e`. `submodules/constitution/Constitution.md` is 11,101 lines; at the time of the run the submodule HEAD matched its pinned gitlink `448981ae3498229c734dc60719f4b19f01d7a75f`.

> ⚠ **That pin no longer holds.** By 18:36Z the submodule had moved to `a09b1ea` and gone dirty while HEAD still pins `448981a` — so `I2 PINNED-REVISION` would now fail. See §5.6.

5.3 Hardcoded-path audit

`./scripts/audit-hardcoded-paths.sh` — **exit 0**.

```
    allowed  scripts/audit-hardcoded-paths.sh
   no machine-specific hardcoded paths (1 file(s) explicitly
allowed)
```

The single allowlisted file is the detector itself, which necessarily contains the patterns it detects — `.hardcoded-paths-allow` holds exactly that one entry, with the reason written into it: *"Excluding it is not an exemption from the rule."* `$HOME`, `~`, `/etc`, `/usr` and `/tmp` are deliberately **not** flagged; only machine-specific roots are.

The *"18 files, 33 occurrences"* claim reproduces exactly. Replaying the audit's own scope (`git ls-files` minus `docs/`, `_content*`, `_analysis/`, `_tests/evidence/`, `.test-evidence/`, `.superpowers/`, `.ashlrcode/`) at `72dc135^` yields 17 files / 32 lines carrying the literal `<macos-host>/Projects/vasic`, plus `_tools/distribute-helixtranslate.sh` with one

generic `/Volumes/...` that the broader pattern also catches — **18** / **33**. The same scope at `72dc135` and at HEAD yields **1 file** / **1 occurrence**, the detector.

5.4 Backend health

`./scripts/ollama-vulkan-remediation.sh --check` — **exit 0**.

```
   library=cpu - GPU is out of the inference path
   /etc/sysconfig/ollama contains GGML_VK_VISIBLE_DEVICES=-1
   0 i915 faults since ollama started (Thu 2026-08-27 09:44:18
CEST)
    (520 in the last 24h are pre-remediation history)
   batch probe: OK 32/32 distinct
```

Note the check takes **over two minutes** on a loaded host — a 120 s timeout was insufficient.

Also verified: `codegraph version` → **1.6.0**; `bash -lc` yields `DO_NOT_TRACK=1` and `CODEGRAPH_TELEMETRY=0` (the non-interactive-shell gap is genuinely closed); `MANUAL-STEPS.md` exists at the project root.

5.5 The index doctor's current verdict — and why it overstates

`./scripts/lumen-index-doctor.sh` — **exit 1, CORRUPTION DETECTED**:

```
integrity_check: ok
files: 2016 fully indexed, 95 queued placeholders | chunks: 52516
vectors: 87040 total, 85300 distinct
   43 duplicate-vector group(s); 1783 vectors (2.05%) are not
unique
   largest identical group: 943 copies of ONE vector
per-vector: 0 NaN/Inf, 1 all-zero, 0 off-norm, 0 ragged block(s)
```

Those numbers are misleading, and I established why. The sqlite-vec store holds 85 blocks × 1024 = **87,040 allocated slots**, but `vec_chunks_chunks.validity` — the bitmap that says which slots are live — marks exactly **52,516**, which equals the chunk count precisely. The doctor decodes all 87,040 and never reads the bitmap, so **34,524 dead slots are counted as stored vectors.**

Recomputing with the bitmap applied:

Population	Slots	Duplicate groups	Duplicated vectors	Per-vector faults
LIVE (real stored vectors)	52,516	**1**	**169 (0.322%)**	0 NaN/Inf, **0 all-zero**, 0 off-norm
DEAD (freed / uninitialised)	34,524	2	1,532	—

- The doctor's *"largest group: 943"* is **943 all-zero uninitialised slots**. Its *"1 all-zero"* is that same dead artifact. Neither is stored data.
- The other 40 of its 43 "groups" are a live vector counted together with its own stale ghost in a freed slot.

What the single live group actually is — and it is real corruption. The 169 vectors resolve to **12 distinct files, 63 distinct symbols, 138 distinct line-spans**, entirely within `_content_fa/` and `_content_es/`. Different headings, different line ranges, two different languages, one identical vector. That is failure mode 4, not boilerplate.

And it is provably the *original* fault, not new damage. Fingerprinting the vector:

Property	Measured now	`INDEX-CORRUPTION-RECONCILIATION.md`
L2 norm	**1.000000083**	1.000000083
Component range	**−0.106444 … +0.107006**	−0.106 … +0.107
Distinct components	**768**	768

The clincher — counting that exact vector across both populations:

```
LIVE  sha256[:16] = 66f94228aff02051  count = 169
DEAD  sha256[:16] = 66f94228aff02051  count = 589
                                      -----------
                                      169 + 589 = 758   ← the
documented original corruption
```

So: of the 758 originally-corrupt vectors, the CPU rebuild has already repaired 589 (77.7%); 169 (22.3%) remain live. No new corruption has been introduced since the backend fix — the only live duplicate group is the original stale vector, byte-for-byte unchanged. The repaired 589 now sit in freed slots, which is exactly why the doctor's unmasked "largest group" appeared to *grow* from 758 to 943 while the backend was healthy.

The doctor's **verdict is correct and its exit code is right**; only its magnitude is wrong. Two fixes are indicated (neither made — this report changed no code): mask by `vec_chunks_chunks.validity`, and count all-zero/NaN by *occurrence* rather than by distinct byte-pattern.

5.6 The tree moved under this report

`git status --short` shows **14 modified/deleted tracked paths and 2 untracked files** — uncommitted work by a concurrent actor, none of it covered by the ledger:

```
 M .github/workflows/ci.yml             (+30/-6 vs HEAD)
 M scripts/setup-agents-wizard.sh       (34 lines changed vs HEAD)
 M scripts/test-setup-agents-wizard.sh (+28 vs HEAD)
 M _tools/audit-hardcoding.sh
 M .gitignore, _analysis/IMPLEMENTATION-REPORT.md, 6 × .ashlrcode/
genome/*
?? docs/setup-agents-wizard/CI-WORKFLOW-AUDIT.md      (209 lines)
?? docs/setup-agents-wizard/RESIDUAL-PATHS-AUDIT.md   (396 lines)
```

And it kept moving while this report was being written. A second check at 18:36Z found three further untracked files that did not exist at 18:26Z, plus another `ci.yml` revision:

Appeared	What it is
`scripts/verify-governance-cascade.sh` (736 lines, 20:36)	**The §11.4.32 step-1 verifier whose absence *is* OC-3.** See §6.3.
`docs/constitution-adoption/REMOTE-ASYMMETRY.md` (271 lines, 20:32)	An OC-2 follow-up on the `milosvasic.ru` fetch/push URL asymmetry
`docs/setup-agents-wizard/LUMEN-STORE-INVENTORY.md` (1,625 lines, 20:34)	A read-only inventory of `~/.local/share/lumen/`. It opens with *"STOP — four indexers are writing RIGHT NOW"* — **independently confirming the contention I measured in §6.1**
`.github/workflows/ci.yml`	

Appeared	What it is
	10,609 B @20:05 → **12,421 B @20:29**; md5 `29cfcdba…` → `9b17930e…`
`submodules/constitution`	gitlink moved off its pin: `448981a` → **`a09b1ea`, and dirty**

The submodule move breaks an invariant that was green when I measured it. At 18:2xZ `tests/test_constitution_inheritance.sh` reported

`PASS I2 PINNED-REVISION — submodules/constitution HEAD == pinned gitlink 448981ae…`. The working tree now has that submodule at `a09b1ea-dirty` while HEAD still pins `448981a`, so **I2 would now FAIL and the inheritance test would no longer be 8/8.** Whoever moved it needs to either commit the bump deliberately or restore the pin — an uncommitted, dirty governance submodule is the one state the constitution's own cascade rules are designed to prevent.

Measured drift in the two files I was instructed not to edit:

File	Earlier this session	At 18:23Z / 18:26Z
`scripts/setup-agents-wizard.sh`	60,263 B, mtime 18:41	**60,751 B, mtime 20:05:44**, stable since
`scripts/test-setup-agents-wizard.sh`	48,535 B, mtime 18:41	**50,074 B, mtime 20:06:29**
`.github/workflows/ci.yml`	—	**10,609 B, mtime 20:05:58**, stable since

Consequence for the headline number: the 147/147 evidence run was written at **19:36 CEST** and therefore measured a wizard **~488 bytes smaller** than the one now on disk. The suite is green *against the version it ran on*, not necessarily against the working tree. Re-run it before relying on the figure.

The two new audits already contradict committed documentation: `CI-WORKFLOW-AUDIT.md` grades 46 claims in `ci.yml` as **3 FALSE** / **5 STALE** / **4 UNVERIFIABLE** / **34 VERIFIED**, and `RESIDUAL-PATHS-AUDIT.md` finds **1,604 tracked files** / **10,424 occurrences** of `/Volumes` still present — with **0 load-bearing**, confirming the executable fix is real, but **6 genome files that still assert the bug is unfixed**, which will actively mislead future agents.

The in-flight `ci.yml` edit is a net improvement and worth recording: it replaces the stale *"Non-invasive and fully honest"* symlink comment (honesty-ledger row 25) with `# ---- Path resolution (the former / Volumes symlink bridge is GONE)`, and promotes the hardcoded-path audit to a real fail-fast **Gate 0**:

```
      - name: Gate — hardcoded path audit
        run: ./scripts/audit-hardcoded-paths.sh
```

None of it is committed, and none of it is covered by the ledger. **Anything this report says about `ci.yml`, `setup-agents-wizard.sh` or `test-setup-agents-wizard.sh` is true of a working tree that was still being edited — re-read them before acting.**

---

6. What is not done

6.1 The Lumen rebuild

`./scripts/lumen-reindex.sh <proj> --force`, started **09:48:16**, still running at **10 h 40 m** elapsed (PIDs 839709 / 840271 / 840272). It is on its **first round** — `.lumen-reindex.log` has had no new line since `09:48:31`, and the script only logs between rounds.

I could not reproduce the ledger's throughput figure. Four samples of `SELECT COUNT(*) FROM files WHERE hash<>''`:

Sample	UTC	Fully indexed	Chunks
1	18:21:30Z	2,016	52,516
2	18:26:26Z	2,016	52,516
3	18:27:02Z	2,016	52,516
4	18:28:50Z	2,016	52,516

Zero measurable forward progress over 7 min 20 s, against a ledger claim of ~15 files/min. The ledger's own close-out snapshot (~17:35Z) recorded 1,995, so roughly **+21 files in ~45 minutes**.

It is **not stalled** — the ollama runner has burned **1 d 17 h 56 m of CPU at 395%**. It is **starved**. Four `lumen index` jobs are competing for one single-slot ollama backend (`vasic`, `lava`, `boba`, `boba/constitution`), on an 8-core host with a load average of **17.8**. This was independently confirmed while the report was being written: the new `docs/setup-agents-wizard/LUMEN-STORE-INVENTORY.md` opens with *"STOP — four*

indexers are writing RIGHT NOW… The brief warned about one active rebuild. There are in fact four concurrent `lumen index` processes." Progress may also be committed in batches rather than continuously; I did not establish which. **The "roughly 2 more hours" estimate should not be relied on.** Reduce the contention or accept that the ETA is unknown.

The remaining work is also uncertain in size — see honesty ledger row 4 (2,413 vs ~3,800).

Still to do when it finishes: re-run `./scripts/lumen-index-doctor.sh`. That is the only thing that settles it. Expect it to report corruption until the last of the 169 live stale vectors is overwritten — **and read its output with §5.5 in hand**, because unmasked it will keep counting freed slots.

6.2 The constitution gates: 37 PASS / 21 FAIL

`GATE-TRIAGE.md`'s categorisation is unambiguous: **zero of the 21 are fixable in the umbrella.** *"There is no umbrella-side change that flips any of the 21."*

- **Rows 1–17** are the 17 `cm_covenant_114_*_propagation.sh` gates. Each is held FAIL by the **same set of carriers simultaneously**, so clearing one flips nothing. After the propagation apply, **4 remain, all third-party:**

Carrier	Owner	Why it cannot be cleared here
`milosvasic.ru/Upstreamable/AGENTS.md`	`red-elf/Upstreamable`	A gitlink inside a submodule; the `red-elf` namespace is not one this project pushes to
`milosvasic.ru/Upstreamable/CLAUDE.md`	`red-elf/Upstreamable`	Same. Also carries a self-referential gitlink with a **0-byte `.gitmodules`**, which makes `git submodule status --recursive` abort rc=128
`submodules/superspec/examples/`	`WangX0111/superspec`	A 5-line SpecKit demo fixture, only

Carrier	Owner	Why it cannot be cleared here
`static-landing-page/CLAUDE.md`		*named* like a governance carrier
`.specify/extensions/superspec/examples/.../CLAUDE.md`	vendored copy of the above	Physically writable, **deliberately not written** — editing it forks the vendored artifact and invalidates the pinned `manifest_hash`

`POST-APPLY-STATE.md`: *"Zero of the four can be cleared by any write into a repository the user owns."* The durable remedy is upstream, in the gate family itself (§11.4.26). **37/21 is the honest ceiling, and it is not a defect in this session's work.**

- **Rows 18/19** are inside `submodules/constitution` (a project-specific literal in `continuum`; a gate-ledger ratchet where `unimplemented=432` exceeds a checked-in `baseline=420`).
- **Rows 20/21** (`cm_track_branch_label*`) are unconditional drift inside the constitution submodule: the labeler emits more fields than it used to, while the gate's `_label_alias()` still extracts the **last** field via `${_p##* - }` — so it reads the *effort* (`xhigh`) where it expects the *alias* (`cmgatechk`). **`POST-APPLY-STATE.md` §6 corrects `GATE-TRIAGE.md` twice here:** the emitted label has **four** `--`-separated fields, not five; and the one-line fix is **necessary but not sufficient** — the gate's own mutation probe (`_head="${_correct% - *}"`) mutates the effort slot, so fixing `_label_alias` alone makes it reachable and the gate then falsely accuses a provably-correct hook. **Two lines, both inside `submodules/constitution`**, which fans out to 6 push URLs.

6.3 Open conflicts — OC-1, OC-2, OC-3

Recorded in `Constitution.md` and mirrored in `docs/constitution-adoption/README.md` §7. All three are deliberately filed as **open conflicts, not overrides**: *"an override is a decision; the items below are undecided contradictions, and recording them as overrides would claim an authorization nobody has given."*

> **SUPERSEDED for OC-1 and OC-2 — decided 2026-08-27, after this report was written.** The table below is retained as the state at report time. Operator decision, verbatim: ***"Comply — disable both, enforce locally."*** ~~Both `.github/workflows/ci.yml` and `milosvasic.ru/.github/workflows/pages.yml` are renamed to a non-active `.disabled` name per §11.4.156(B)~~ — **`.github/workflows/ci.yml` is** — and the gates move to a **local pre-push hook**. **Decision option (2) in the OC-1 row below — *"record an explicit `Override §11.4.156`"* — does not exist**: §11.4.156's closing formula names and refuses the exemption vocabulary (*"No escape hatch … `--ci-exempt`"*), and the inheritance contract (*"extend them — they do NOT weaken or override any universal clause"*) makes a project-local override structurally impossible. Compliance was the only remaining path that does not amend shared governance; no amendment was made. **Honest boundary (§11.4.6):** the rename stops FILE-triggered runs only. Provider-side settings — org-default required workflows, branch-protection required checks, provider-side scheduled exports — are operator-only manual steps and are unverified, so G4 moves to **PARTIAL, not CLOSED**. **Cost:** no server-side enforcement on push or PR; `.git/hooks/` is not tracked by git, so a fresh clone has **no** protection until `bash scripts/pre-push-gates.sh --install` is run, and `git push --no-verify` bypasses the hook. Full record: `constitution-adoption/DECISION-11-4-156-COMPLY.md`. **OC-3 is unaffected by this decision.**

> ⚠ **PARTIAL REVERSAL, same day — read this with the banner above.** The *"disable both"* decision applies to `ci.yml` **only**. `milosvasic.ru/.github/workflows/pages.yml` was **restored to ACTIVE and will stay active**. Material fact: `gh api repos/milos85vasic/milosvasic.ru/pages` returns `build_type: "workflow"` — GitHub Pages publishes the live production site `https://milosvasic.ru/` *exclusively* by running that workflow. There is no `gh-pages` branch, no `docs/` folder, and the repository root is Jekyll SOURCE (Liquid + front matter), so it cannot be served raw from a branch. `_tools/deploy-langs.sh` is **not** a substitute: it generates, commits and pushes source, then `sleep`s waiting for the server to rebuild — it covers none of the publish step. Disabling `pages.yml` would end publishing, not make it manual. **Operator's overriding directive, verbatim: *"Make sure all pages websites work flawlessly! No website can be broken! All websites we have here are running deployed in production!"*** **Net state:** the umbrella root complies with §11.4.156 at file level; **`milosvasic.ru` does not and will not — a**

> **known, documented deviation, emphatically NOT an `Override §11.4.156`** (the rule forbids one). Additionally **`vasic.digital` is non-compliant at the provider level with no file-level remedy**: `build_type: "legacy"` triggers a `pages build and deployment` Actions run on every push although the repository contains **zero** workflow files. See `constitution-adoption/DECISION-11-4-156-COMPLY.md` §0.

ID	Conflict	Why it is unresolved	Decision needed
OC-1 *(⚠ DECIDED 2026-08-27 — see banner; option 2 does not exist)*	`.github/workflows/ci.yml` is live (`on: push`, `on: pull_request`) while §11.4.156(A) forbids **any** active CI at a governed repo root — *"a release blocker regardless of context. No escape hatch."*	It is a conflict of **intent, not neglect**. §11.4.156 moves enforcement to the §11.4.75 five-layer git-hook ritual, and this repo has **none** of those layers (gap G5). Disabling CI today deletes the only mechanical check and puts nothing in its place.	One of three, none of which an agent may pick: (1) disable per §11.4.156(B) **and** stand up the §11.4.75 layers first; (2) record an explicit `Override §11.4.156` with justification; (3) keep both and knowingly accept the release-blocker state. ⚠ `Constitution.md`'s "9,526 bytes" for this file is **stale** — it is now 10,609 B.
OC-2 *(⚠ DECIDED 2026-08-27 — see banner)*	`milosvasic.ru/.github/workflows/pages.yml` — the same clause, inside an **owned** submodule	Listed separately because resolving it means committing **inside a submodule working tree**, which §102 reserves to the operator. (`submodules/superspec/.github/workflows/ci.yml`	Operator commit inside the submodule

ID	Conflict	Why it is unresolved	Decision needed
		is deliberately *not* listed: §11.4.156(C) scopes to repositories *"we author + push"*, and superspec is third-party.)	
OC-3	§11.4.32 step 1 requires re-running `scripts/verify-governance-cascade.sh`, which did not exist here	The sweep reports step 1 as an explicit SKIP naming the missing file, and never as a pass — correct behaviour, but the step is genuinely unperformed.	⚠ **Being closed as this was written.** A 736-line `scripts/verify-governance-cascade.sh` appeared **untracked** at 20:36, after my sweep run. My 37/21 measurement therefore predates it and still shows step 1 as SKIP. **Re-run the sweep**; OC-3 may already be resolved, but the new script is uncommitted, unreviewed, and was not exercised by any measurement in this report.

No fourth `OC-` identifier exists.

6.4 Everything else the ledger or the artifacts flag as pending

- **`WOZCODE`** has no public installer. Reported honestly as `n/a` rather than , with `WOZCODE_INSTALL_CMD` documented as the enable path. **Needs operator input.**
- **The glyphdown hook** is held back on operator request — a deliberate skip, not a gap.
- **`I11` and `I12`** are still source greps that never execute what they name (§3.1), and `I12` is *documented* as behavioural. `A48/A49/A50` and `I16–I20` are the same shape. Pass 2's recommendation 5 — make them execute the scripts with stubs — is **not done**.
- **`C-3` is acknowledged but unfixed:** the test-suite header still reads *"Nothing here mutates the real environment"* (I verified the line is unchanged), while the suite writes `.test-evidence/<stamp>/` into the repo and live mode runs real `lumen index`/`purge`.
- **`G10`** still inspects one file to prove a whole-filesystem property.
- **The `cm_opendesign_ui_system.sh` false SKIP** is knowingly left in place. Binding `OD_THEME_GLOBS` converts it to an honest FAIL and surfaces **187 hardcoded six-digit hex literals** across five theme files. Operator call.
- **`Constitution.md`'s "Known-excluded gate findings" is stale in three places** — still says five carriers / 85 MISSING lines (now four / 68) and still describes the `vasic.digital/QWEN.md` fix as *"staged, awaits operator application"* when it was applied and pushed as `6e5411c`.
- **`INDEX-COVERAGE.md` is a 2026-08-26 snapshot** and predates the propagation apply; all five submodule SHAs it records are superseded. Its CodeGraph identity (541 indexed == 541 eligible) probably still holds but was measured against a different tree. Its Lumen figures (496 files, zero submodule coverage) are long superseded — I measure **465 submodule files** indexed now.
- **`ARCHITECTURE.md`** was flagged stale mid-session; it has since been rewritten (mtime 18:24), but the 67-finding `DOC-AUDIT.md` (17 Critical / 41 Major / 9 Minor) has no closure record.
- **`submodules/constitution` is ~60 commits behind its real upstream**, with a stale `origin/main` making `git status` read "up to date" — *"a false reassurance."*
- **`.ashlrcode/` is tracked, not gitignored**, and `.ashlrcode/genome/knowledge/discoveries-auto.md` contains raw `{"stdout":...}` dumps that will dilute retrieval against the hand-written `discoveries.md`. A defensible decision, but a decision — and per `RESIDUAL-PATHS-AUDIT.md`, six genome files still assert the hardcoded-path bug is unfixed.

- **The uncommitted workstream in §5.6** is unreviewed and unmerged.

---

7. Operational runbook

The five scripts you reach for, and when.

Script	One line	When to reach for it
`scripts/setup-agents-wizard.sh`	Installs and configures the five agent CLIs, wires Lumen (+CodeGraph) as MCP servers, opts out of telemetry, and ends with an **ACTION REQUIRED** block for what a shell script genuinely cannot do.	First-time setup, or after any agent CLI upgrade. Idempotent; every mutation is recorded to a backup manifest.
`scripts/rollback-agents-wizard.sh`	Replays a wizard run's manifest in reverse — restores byte-exact originals, deletes created files, prints (and with `--run-actions` executes) undo commands.	The wizard changed something you did not want. Start with `--list`, then `--dry-run`, then `--yes`. Rolling back is itself reversible.
`scripts/ollama-vulkan-remediation.sh`	Takes the GPU out of ollama's embedding path. **Defaults to the read-only `--check`.**	**Before every index run**, and first whenever embeddings look wrong. `--apply`/`--rollback` need `sudo`. Allow it >2 min on a loaded host. Do **not** run `--check` against a Vulkan backend while an index is in flight — its probe is large enough to

Script	One line	When to reach for it
		cause the fault it detects.
`scripts/lumen-reindex.sh [path] [--force] [--allow-gpu]`	Rebuilds a Lumen index, **refusing to start** when ollama reports `library=Vulkan`, probing aggregate distinctness first, and resuming incrementally around transient faults.	Any rebuild. **`--force` is mandatory after a corruption event** — corrupted files carry a non-empty hash, so an incremental run skips them forever.
`scripts/lumen-index-doctor.sh [path]`	Detects the corruption class every per-vector test misses: *N* distinct texts must yield *N* distinct vectors. Read-only (`mode=ro`). Exit `0` healthy / `1` corruption / `2` could not inspect.	After every rebuild, and whenever search quality is suspect. **Read its magnitude with §5.5 in hand** — it counts freed slots.

Three more are verification gates rather than operations: `scripts/test-setup-agents-wizard.sh` (the 147-assertion suite; `--no-live` to skip network/daemon), `scripts/audit-hardcoded-paths.sh` (fails on machine-specific absolute paths; CI-suitable), and `scripts/verify-all-constitution-rules.sh` (the §11.4.32 58-gate sweep; expect exit 1 at 37/21 until the third-party carriers are resolved upstream).

Two standing rules earned the hard way this session:

1. **Never `commit` while agents are still writing.** It happened three times; twice it swept artifacts into history and once it nearly pushed a regenerated sitemap to a live site.
2. **The commit wrapper is unsafe in a submodule that lacks its own `upstreams/` directory** — it silently walks up and targets the *parent's* remote names, and still prints `OK`. Verify `local == remote` after every wrapper push.

Prepared 2026-08-27. Measurements taken 18:21Z–18:36Z against HEAD `62706a9`, with a working tree that was being modified by another actor throughout — three new files and two `ci.yml` revisions landed during the writing. Re-measure before quoting any figure here as current. This report created exactly one file (`docs/SESSION-REPORT.md`) and modified none.